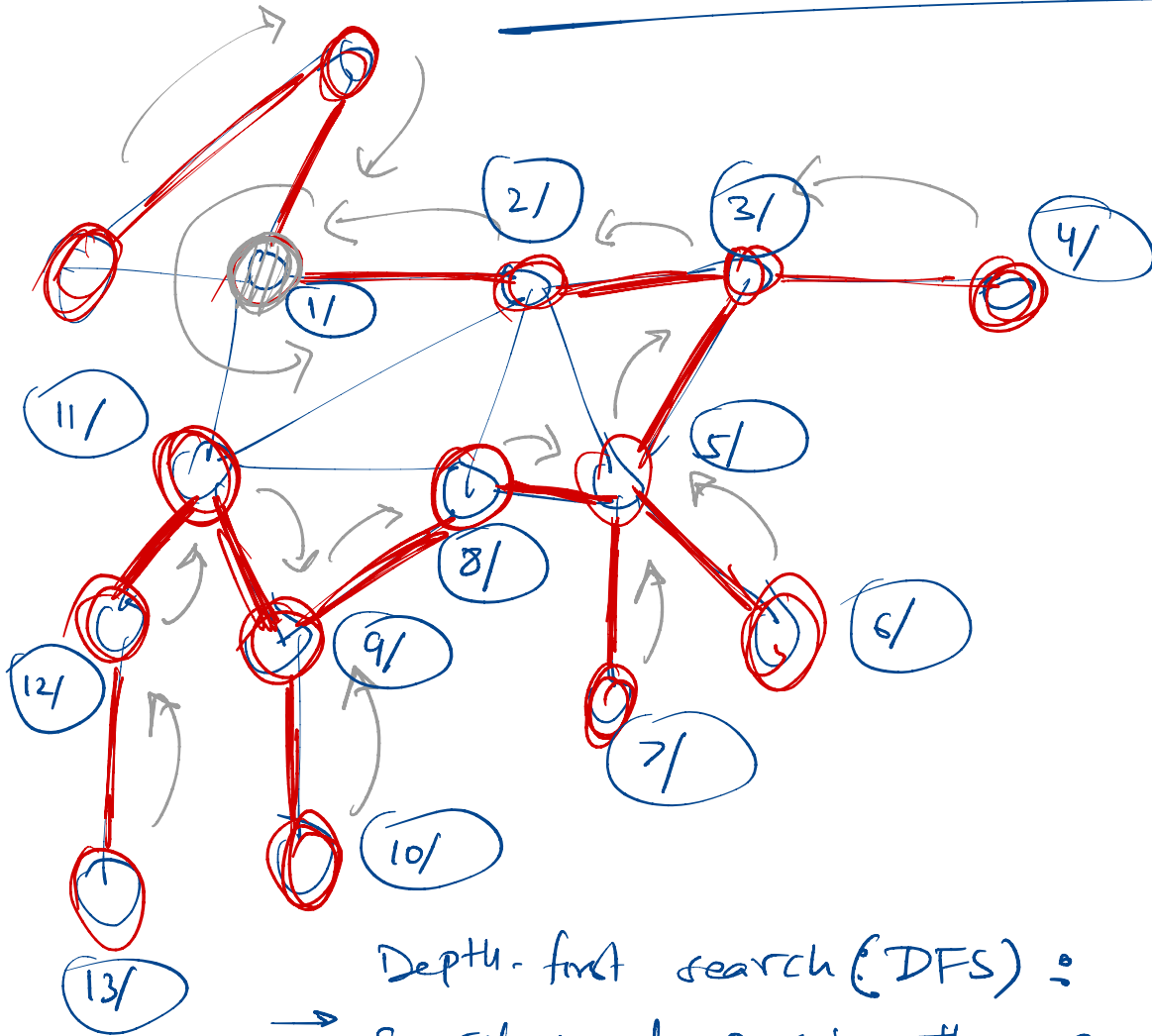


Graph Algorithms - II



Depth-first search (DFS) :

- Searches deeper in the graph whenever possible.
- Explores edges out of the most recently discovered vertex v that still has unexplored edges leaving it.
- Once all of v 's edges have been explored, the search "backtracks" to explore edges leaving the vertex from which v was discovered.

Predecessor Subgraph:

$$G_\pi = (V, E_\pi), \text{ where}$$

$$E_\pi = \{ (v, \pi, u) : v \in V, \text{ and } u, \pi \neq \text{NIL} \}$$

- Depth-first search forms a depth-first forest comprising several depth-first trees.
- Edges in E_π are tree edges.

Depth-first search implementation:

- DFS maintains two timestamps with each vertex:
 - ① $u.d$: when u is first discovered
↳ color it gray then.
 - ② $u.f$: when search finishes examining the vertex (i.e. its adjacency list)
↳ color it black then.
- for every vertex $u.d < u.f$
- Before $u.d \rightarrow$ vertex u is colored WHITE
- Between $u.d$ & $u.f \rightarrow$ vertex u is colored GRAY
- After $u.f \rightarrow$ vertex u is colored BLACK

DFS (G)

for each vertex $u \in G.V$

| $u.color = WHITE$
| $u.\pi = NIL$

time = 0

for each vertex $u \in G.V$

| if $u.color == WHITE$
| | DFS-VISIT (G, u)

$\theta(V+E)$

Complexity

DFS-VISIT (G, u)

time = time + 1

$u.d = time$ $\Rightarrow u$ is discovered

$u.color = GRAY$

for each vertex v in $G.Adj[u]$

| if $v.color == WHITE$

| | $v.\pi = u$

| | DFS-VISIT (G, v)

time = time + 1

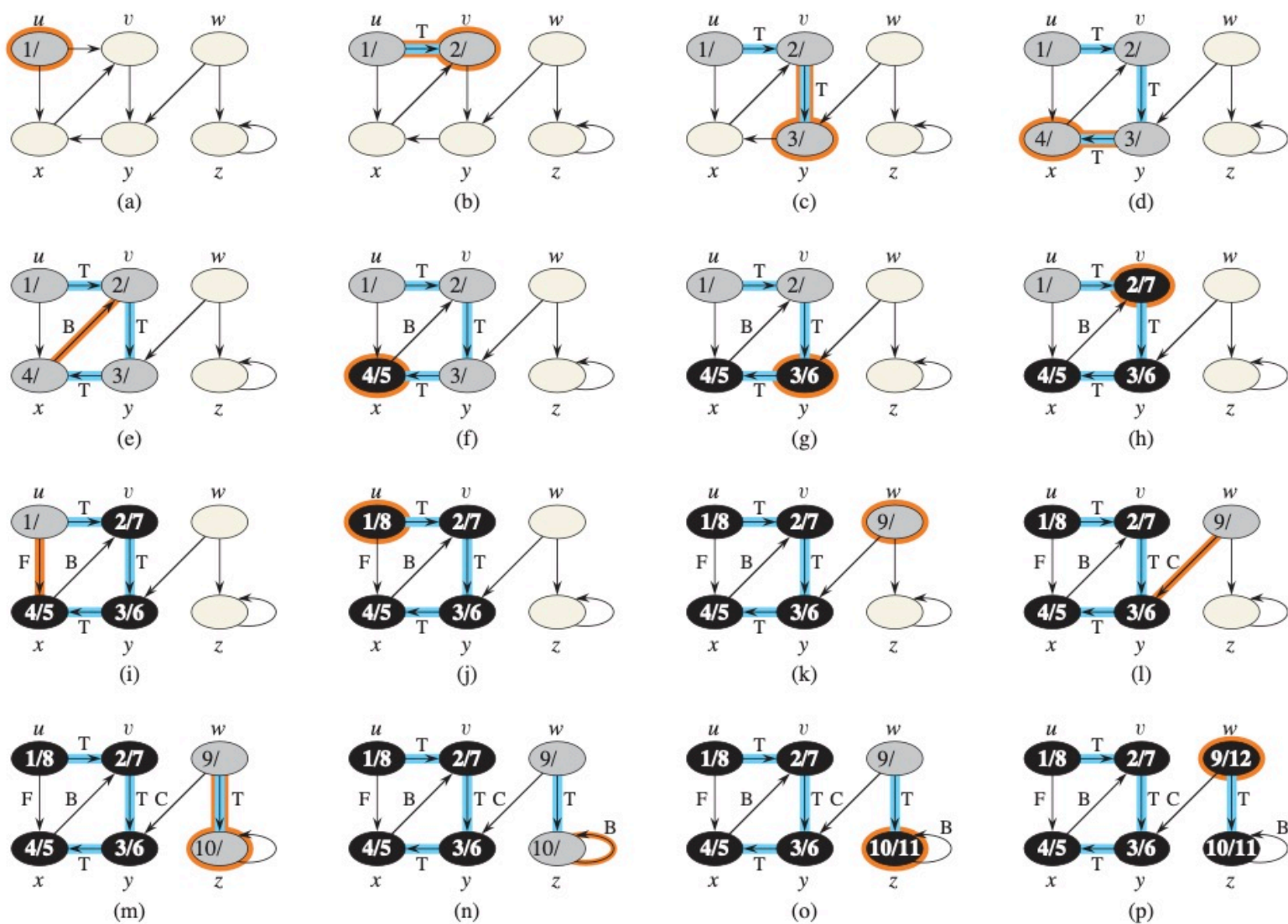
$u.f = time$

$u.color = BLACK$

— Every call of $\text{DFS-VISIT}(G, u)$ makes u the root of a new tree in the depth-first forest.

— Results of depth-first search may depend on the the order of selection of vertices in V and on the order of vertices in the adjacency lists.

Example:

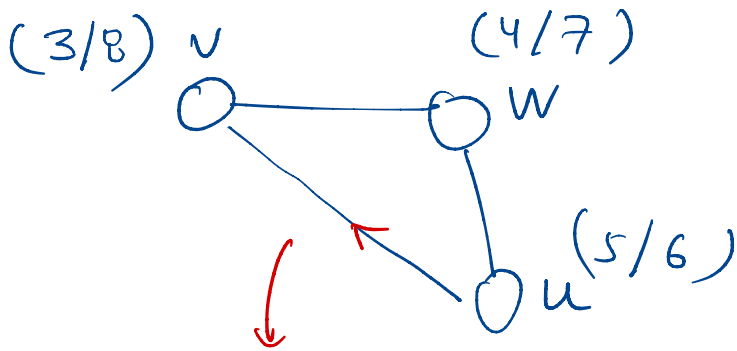


Classification of edges of G

1. Tree edges are edges in G_π . Edge (u, v) is a tree edge if v was first discovered by exploring edge (u, v) . 

Basically any edge that's part of the DF forest.

2. Back edges are those edges (u, v) connecting a vertex u to an ancestor v in the DF tree

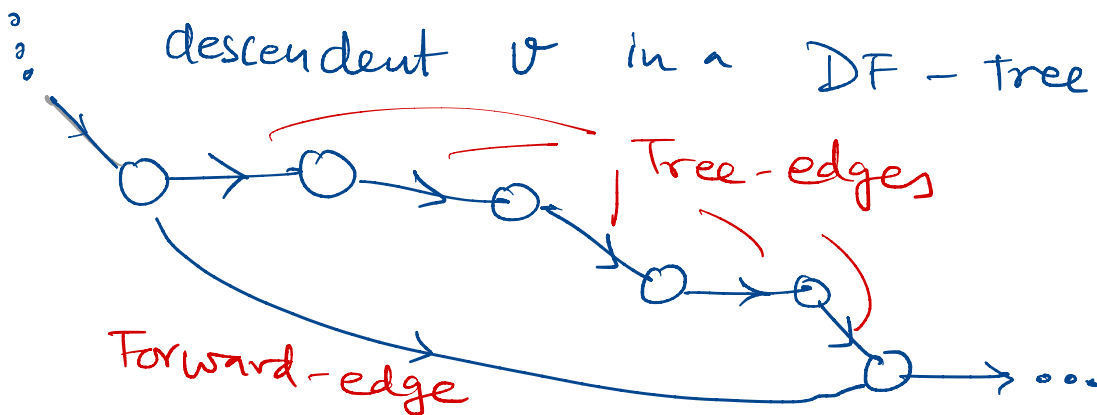


$$u, \pi = w$$

$$\omega \cdot \pi = v$$

Edge (u,v) : Back-edge because v is an ancestor of u

3. Forward edges are those non-tree edges (u, v) connecting a vertex u to a proper descendent v in a DF-tree.



if $u.d < v.d$

4. Cross-edges are all other edges.

They can go between vertices in the same DF tree (as long as one vertex is not an ancestor of another), or between two DF trees.

if $u.d > v.d$

Cycle detection (Application 1 of DFS)

G has a cycle $\iff$ DFS has a back edge.

A given graph G has a cycle provided it has a back edge.

Therefore for detecting cycles in a graph, we just need to check for back edges.

Topological Sort (Application 2 of DFS)

Given a Directed Acyclic Graph (DAG)

↙
No cycles!

$G = (V, E)$, a topological sort

is a linear ordering of its vertices
Such that if G contains an edge (u, v)
then u appears before v in the ordering.

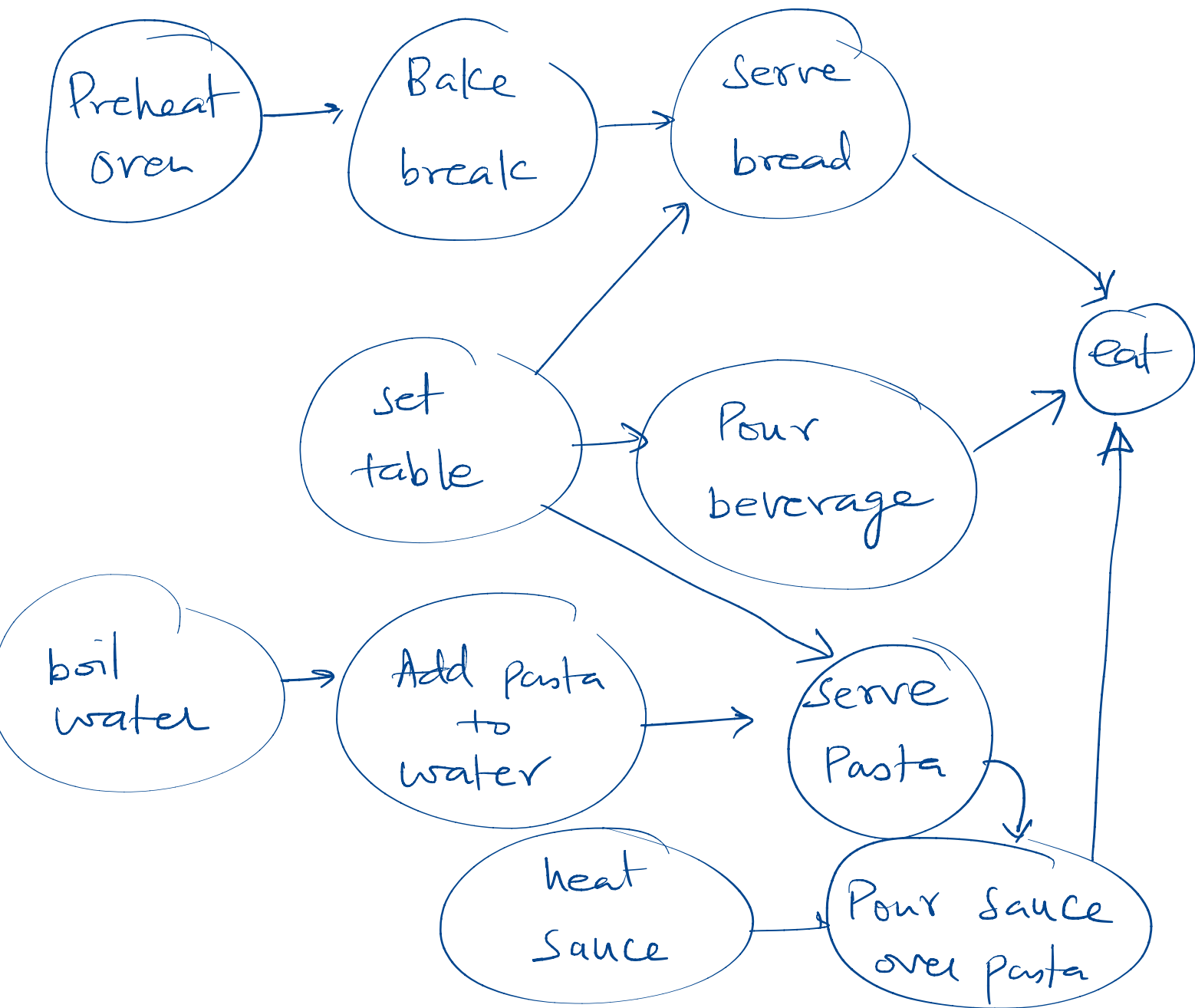
→ is a DAG
Topological Sort (G)

Call DFS(G) to compute finish times $v.f$
for each vertex v

As each vertex is finished, insert it
 $\Theta(V+E)$ onto the front of a linked list

Return the linked list of vertices

Example: Cooking a pasta : as a DAG



Preheat oven

Set table

boil water

bake bread

Add pasta to water

Pour beverage

Serve bread

heat sauce

Serve pasta

Pour sauce over pasta

eat

topological
sort